

Jak działa Internet – TCP i NAT

Witajcie w ostatnim artykule z serii „Jak działa Internet”! Zapoznaliśmy się w niej z różnymi zagadnieniami dotyczącymi Internetu – od najniższych warstw modelu OSI oraz adresowania IP i MAC po protokoły DNS i HTTP. Od czasu do czasu zdarza się jednak, że musimy przyrzeć się warstwie „sklejającej” omówione dotąd mechanizmy. Dlatego w tym artykule poznamy podstawowe zagadnienia związane z protokołami TCP i UDP oraz zapoznamy się z procesem NAT.

I WPROWADZENIE DO TCP ORAZ UDP

Często spotykamy się z nazwą TCP/IP i chociaż nazwa zawiera protokół TCP i IP, to odnosi się do wszystkich omawianych i wspomnianych do tej pory protokołów (HTTP, DNS, ARP) [1]. W tym artykule skupimy się na protokołach TCP oraz UDP, które są częścią warstwy transportowej (czwartej warstwy modelu OSI). To one decydują o sposobie nawiązywania połączenia oraz walidacji poprawności otrzymanych danych.

I UDP

UDP (ang. *User Datagram Protocol*) jest protokołem znacznie prostszym niż TCP. Prostota ta jednak skutkuje ograniczonymi możliwościami. W ramach tego protokołu nie nawiązujemy połączenia ani nie otrzymujemy potwierdzenia dostarczenia pakietu. Korzystając z UDP, musimy liczyć się z tym, że niektóre pakiety mogą zostać utracone, a aplikacje muszą takie straty poprawnie obsłużyć. [5]

Zaletą UDP jest jednak jego „szybkość” w porównaniu do TCP – brak nawiązywania połączenia oraz brak mechanizmu potwierdzenia odbioru pakietów skutkuje mniejszą ilością przesyłanych danych. Dzięki temu jest on szeroko używany chociażby w strumieniowaniu wideo, grach online i czatów głosowych [6].

I TCP

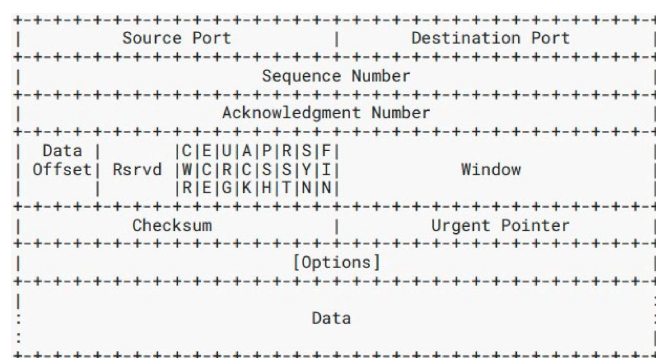
TCP jest niejako przeciwieństwem UDP zarówno co do narzutu danych, jak i funkcjonalności. Komunikacja w protokole TCP zaczyna się od nawiązania połączenia. Zarówno wysłane, jak i otrzymane pakiety są potwierdzane przez obie strony komunikacji. Wszystko to prowadzi do większej ilości wymienianych danych, ale za to mamy większe prawdopodobieństwo wykrycia błędów podczas ich przesyłu [7].

Ze względu na swoją charakterystykę protokół ten jest używany w większości aplikacji do komunikacji z serwisami oraz do transmisji plików. W tych przypadkach ważne jest, aby wiadomość dotarła kompletna bez błędów.

Zarówno protokół TCP, jak i UDP ma tzw. sumę kontrolną (ang. *checksum*). Suma ta zmniejsza prawdopodobieństwo zaakceptowania błędnego pakietu, ale mu nie zapobiega. Dużo lepszą integralność danych gwarantują dopiero protokoły wyższego poziomu jak np. TLS [16].

I SZCZEGÓŁY TCP

Tematyki tego protokołu dotknęliśmy już przy okazji omawiania protokołu HTTP. HTTP w większości przypadków używa właśnie TCP do transmisji danych, dlatego ważne jest dla nas jego głębsze zrozumienie. Na Rysunku 1 przedstawiono strukturę pakietu TCP podaną w rfc9293 (specyfikacji protokołu) [3].



Rysunek 1. Schemat pakietu według RFC 9293

Widzimy wiele pól w pakiecie, a o najważniejszych z nich opowiemy w tym artykule [8]:

Port źródłowy i docelowy (source port, destination port) – to nic innego jak liczby, które pozwalają identyfikować komunikację między różnymi aplikacjami w ramach jednego hosta. Możemy to sobie wyobrazić na przykładzie przeglądarki internetowej z wieloma otwartymi zakładkami, na których jest wczytana ta sama strona internetowa. Ponieważ strona internetowa jest pewną aplikacją, to wszystkie pakiety TCP wysłane przez te zakładki będą miały ten sam port docelowy. Jednak różne zakładki mogą poruszać się po różnych podstronach witryny, żądając innych danych, dlatego każde takie połączenie do serwera będzie mieć inny port źródłowy. Dzięki temu serwer będzie mógł odróżnić połączenia od siebie i wysłać odpowiednie dane, używając odpowiedniego połączenia.

Numer sekwencyjny i numer potwierdzenia (sequence number i acknowledgement number) – liczby pozwalające stronom komunikacji na wzajemne informowanie się o wysłanych i odebranych danych, co umożliwia retransmisję zagubionych pakietów.

Data offset – informuje nas, gdzie w pakiecie znajdują się dane przeznaczone dla aplikacji. Wartość ta jest zmienna ze względu na pole Options, które ma zmienną długość.

Flagi (CWR, ECE, URG, ACK, RST, PSH, SYN, FIN) – bity określające przeznaczenie danego pakietu. Na przykład flaga SYN służy do nawiązania połączenia (synchronizacji), a PSH (push) do przesyłania danych.

Okno (ang. Window) – Informuje o wielkości bufora danych zarezerwowanego do komunikacji między klientem a serwerem. Jego użycie w praktyce zobaczymy w dalszej części artykułu.

Checksum – suma kontrolna pozwalająca na weryfikację poprawności przesłanych danych.

Urgent Pointer – obecnie jest głównie nieużywany. Nie będzie omawiany w tym artykule.

Options – dodatkowe opcje połączenia, które mogą być wspierane przez strony komunikacji. Pole może również zawierać dodatkowe dane przesyłane razem z pakietem, jak np. te omawiane w sekcji SACK.

Data – dane wysyłane do aplikacji w pakiecie TCP.

Spójrzmy na pakiety TCP w akcji. Wyślijmy żądanie HTTP do dowolnego serwisu (w tym przypadku <http://www.google.com>) i podejrzmy pakiety w aplikacji Wireshark.

Uruchommy więc Wireshark, a do wysłania żądania wykorzystamy narzędzie cURL, jak przedstawiono to w Listingu 1.

Listing 1. Wykorzystanie narzędzia cURL do wysłania żądania

```
curl http://www.google.com
```

W oknie Wiresharka pojawiają się nam kolejne pakiety TCP. Po ich odfiltrowaniu powinniśmy ujrzeć okno podobne do tego z Rysunku 2. Na jego podstawie przeanalizujemy przebieg komunikacji TCP.

Nawiązywanie połączenia TCP

Na początku komunikacji następuje proces nawiązywania połączenia. Często określa się go jako *three-way handshake*, ponieważ składa się z trzech wymienianych pakietów (klient → serwer, serwer → klient, klient → serwer). Warto wspomnieć, że jeden z tych pakietów wynika z połączenia pakietów SYN i ACK, które zobaczymy za chwilę. W związku z tym poprawne jest również nawiązywanie połączenia w czterech krokach.

Protokół TCP przewiduje także sytuację, w której obie strony jednocześnie inicjują nawiązanie połączenia. Chociaż jest to rzadkie zjawisko, warto o nim pamiętać, analizując mniej typowe scenariusze komunikacji TCP [9].

Korzystając z opcji Flow Graph (dostępnej w Statistics → Flow Graph), możemy lepiej zwizualizować przebieg komunikacji, co pokazuje Rysunek 3.

No.	Time	Source	Destination	Protocol	Length	Info
44	8.304333877	10.11.0.25	216.239.38.120	TCP	74	58250 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1529650600 TSecr=0 WS=128
45	8.319159675	216.239.38.120	10.11.0.25	TCP	74	80 → 58250 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM TSval=2049743639 TSecr=1529650600 WS=256
46	8.319278576	10.11.0.25	216.239.38.120	TCP	66	58250 → 80 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=1529650615 TSecr=2049743639
47	8.319553379	10.11.0.25	216.239.38.120	HTTP	144	GET / HTTP/1.1
48	8.337382260	216.239.38.120	10.11.0.25	TCP	66	80 → 58250 [ACK] Seq=1 Ack=79 Win=268800 Len=0 TSval=2049743658 TSecr=1529650615
49	8.810539294	216.239.38.120	10.11.0.25	TCP	2866	80 → 58250 [PSH, ACK] Seq=1 Ack=79 Win=268800 Len=2800 TSval=2049744091 TSecr=1529650615 [TCP PDU reassembled in 6]
50	8.810539931	216.239.38.120	10.11.0.25	TCP	2866	80 → 58250 [PSH, ACK] Seq=2801 Ack=79 Win=268800 Len=2800 TSval=2049744091 TSecr=1529650615 [TCP PDU reassembled in 6]
51	8.810540214	216.239.38.120	10.11.0.25	TCP	2866	80 → 58250 [PSH, ACK] Seq=5601 Ack=79 Win=268800 Len=2800 TSval=2049744091 TSecr=1529650615 [TCP PDU reassembled in 6]
52	8.810540538	216.239.38.120	10.11.0.25	TCP	2866	80 → 58250 [PSH, ACK] Seq=8401 Ack=79 Win=268800 Len=2800 TSval=2049744091 TSecr=1529650615 [TCP PDU reassembled in 6]
53	8.810709524	10.11.0.25	216.239.38.120	TCP	66	58250 → 80 [ACK] Seq=79 Ack=2801 Win=69760 Len=0 TSval=1529651107 TSecr=2049744091
54	8.810787500	10.11.0.25	216.239.38.120	TCP	66	58250 → 80 [ACK] Seq=79 Ack=5601 Win=75392 Len=0 TSval=1529651107 TSecr=2049744091
55	8.810887287	10.11.0.25	216.239.38.120	TCP	66	58250 → 80 [ACK] Seq=79 Ack=11201 Win=78848 Len=0 TSval=1529651107 TSecr=2049744091
56	8.810942246	216.239.38.120	10.11.0.25	TCP	2866	80 → 58250 [PSH, ACK] Seq=11201 Ack=79 Win=268800 Len=2800 TSval=2049744091 TSecr=1529650615 [TCP PDU reassembled in 6]
57	8.810942836	216.239.38.120	10.11.0.25	TCP	1466	80 → 58250 [ACK] Seq=14001 Ack=79 Win=268800 Len=1400 TSval=2049744125 TSecr=1529650615 [TCP PDU reassembled in 6]
58	8.811005176	10.11.0.25	216.239.38.120	TCP	66	58250 → 80 [ACK] Seq=79 Ack=14001 Win=81664 Len=0 TSval=1529651107 TSecr=2049744091

Rysunek 2. Podgląd komunikacji TCP w aplikacji Wireshark

Time	10.11.0.25	216.239.38.120	Comment
8.304333877	58250 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1529650600 TSecr=0 WS=128	80	TCP: 58250 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1529650600 TSecr=0 WS=128
8.319159675	80 → 58250 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM TSval=2049743639 TSecr=1529650600 WS=256	80	TCP: 80 → 58250 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM TSval=2049743639 TSecr=1529650600 WS=256
8.319278576	58250 → 80 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=1529650615 TSecr=2049743639	80	TCP: 58250 → 80 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=1529650615 TSecr=2049743639
8.319553379	GET / HTTP/1.1	80	HTTP: GET / HTTP/1.1
8.337382260	80 → 58250 [ACK] Seq=1 Ack=79 Win=268800 Len=0 TSval=2049743658 TSecr=1529650615	80	TCP: 80 → 58250 [ACK] Seq=1 Ack=79 Win=268800 Len=0 TSval=2049743658 TSecr=1529650615
8.810539294	80 → 58250 [PSH, ACK] Seq=1 Ack=79 Win=268800 Len=2800 TSval=2049744091 TSecr=1529650615 [TCP PDU reassembled in 6]	80	TCP: 80 → 58250 [PSH, ACK] Seq=1 Ack=79 Win=268800 Len=2800 TSval=2049744091 TSecr=1529650615 [TCP PDU reassembled in 6]
8.810539931	80 → 58250 [PSH, ACK] Seq=2801 Ack=79 Win=268800 Len=2800 TSval=2049744091 TSecr=1529650615 [TCP PDU reassembled in 6]	80	TCP: 80 → 58250 [PSH, ACK] Seq=2801 Ack=79 Win=268800 Len=2800 TSval=2049744091 TSecr=1529650615 [TCP PDU reassembled in 6]
8.810540214	80 → 58250 [PSH, ACK] Seq=5601 Ack=79 Win=268800 Len=2800 TSval=2049744091 TSecr=1529650615 [TCP PDU reassembled in 6]	80	TCP: 80 → 58250 [PSH, ACK] Seq=5601 Ack=79 Win=268800 Len=2800 TSval=2049744091 TSecr=1529650615 [TCP PDU reassembled in 6]
8.810540538	80 → 58250 [PSH, ACK] Seq=8401 Ack=79 Win=268800 Len=2800 TSval=2049744091 TSecr=1529650615 [TCP PDU reassembled in 6]	80	TCP: 80 → 58250 [PSH, ACK] Seq=8401 Ack=79 Win=268800 Len=2800 TSval=2049744091 TSecr=1529650615 [TCP PDU reassembled in 6]
8.810709524	58250 → 80 [ACK] Seq=79 Ack=2801 Win=69760 Len=0 TSval=1529651107 TSecr=2049744091	80	TCP: 58250 → 80 [ACK] Seq=79 Ack=2801 Win=69760 Len=0 TSval=1529651107 TSecr=2049744091
8.810787500	58250 → 80 [ACK] Seq=79 Ack=5601 Win=75392 Len=0 TSval=1529651107 TSecr=2049744091	80	TCP: 58250 → 80 [ACK] Seq=79 Ack=5601 Win=75392 Len=0 TSval=1529651107 TSecr=2049744091
8.810887287	58250 → 80 [ACK] Seq=79 Ack=11201 Win=78848 Len=0 TSval=1529651107 TSecr=2049744091	80	TCP: 58250 → 80 [ACK] Seq=79 Ack=11201 Win=78848 Len=0 TSval=1529651107 TSecr=2049744091
8.810942246	80 → 58250 [PSH, ACK] Seq=11201 Ack=79 Win=268800 Len=2800 TSval=2049744091 TSecr=1529650615 [TCP PDU reassembled in 6]	80	TCP: 80 → 58250 [PSH, ACK] Seq=11201 Ack=79 Win=268800 Len=2800 TSval=2049744091 TSecr=1529650615 [TCP PDU reassembled in 6]
8.810942836	80 → 58250 [ACK] Seq=14001 Ack=79 Win=268800 Len=1400 TSval=2049744125 TSecr=1529650615 [TCP PDU reassembled in 6]	80	TCP: 80 → 58250 [ACK] Seq=14001 Ack=79 Win=268800 Len=1400 TSval=2049744125 TSecr=1529650615 [TCP PDU reassembled in 6]
8.811005176	58250 → 80 [ACK] Seq=79 Ack=14001 Win=81664 Len=0 TSval=1529651107 TSecr=2049744091	80	TCP: 58250 → 80 [ACK] Seq=79 Ack=14001 Win=81664 Len=0 TSval=1529651107 TSecr=2049744091
8.811048511	58250 → 80 [ACK] Seq=79 Ack=15401 Win=80384 Len=0 TSval=1529651107 TSecr=2049744125	80	TCP: 58250 → 80 [ACK] Seq=79 Ack=15401 Win=80384 Len=0 TSval=1529651107 TSecr=2049744125
8.826482235	80 → 58250 [PSH, ACK] Seq=15401 Ack=79 Win=268800 Len=2800 TSval=2049744145 TSecr=1529651107 [TCP PDU reassembled in 6]	80	TCP: 80 → 58250 [PSH, ACK] Seq=15401 Ack=79 Win=268800 Len=2800 TSval=2049744145 TSecr=1529651107 [TCP PDU reassembled in 6]
8.826483077	80 → 58250 [PSH, ACK] Seq=18201 Ack=79 Win=268800 Len=2800 TSval=2049744146 TSecr=1529651107 [TCP PDU reassembled in 6]	80	TCP: 80 → 58250 [PSH, ACK] Seq=18201 Ack=79 Win=268800 Len=2800 TSval=2049744146 TSecr=1529651107 [TCP PDU reassembled in 6]
8.826483404	HTTP/1.1 200 OK (text/html)	80	HTTP: HTTP/1.1 200 OK (text/html)

Rysunek 3. Komunikacja TCP w widoku Flow Graph

W szczególności przyjrzyjmy się wspomnianemu nawiązywaniu połączenia przedstawionemu na Rysunku 4.

Widać tutaj wspomniany three-way handshake. W szczególności warto zwrócić uwagę na flagi pakietów – w tym przypadku są to SYN oraz ACK, co można zobaczyć na rysunku. Flagi określają przeznaczenie pakietu: SYN pochodzi od słowa synchronize, a ACK od acknowledge.

Pakiety SYN służą głównie do nawiązania połączenia. Klient i serwer wymieniają się parametrami połączenia, takimi jak początkowe wartości sekwencyjne, rozmiar okna i jego skalowanie oraz informacjami o wspieranych rozszerzeniach protokołu TCP (np. wsparcie dla SACK [10] omawiane w dalszej części artykułu). Te parametry będą używane do interpretacji kolejnych pakietów i zapewnienia poprawnej komunikacji.

Warto dodać, że różne zrzuty ekranu z aplikacji Wireshark mogą mieć nieco modyfikowany widok w zależności od tego, na czym się skupiamy. Ponadto niektóre rysunki mogą pochodzić z różnych połączeń TCP podobnego żądania HTTP.

Widok w Wiresharku łatwo można dostosować np. poprzez dodanie nowych kolumn do listy pakietów. W tym celu należy kliknąć prawym przyciskiem myszy interesującą nas informację z widoku i wybierać opcję „Apply As Column” dostępną po naciśnięciu prawego przycisku myszy.

Sequence number i ACKnowledgement

Przyjrzyjmy się wartościom liczb sekwencyjnych i ich potwierdzeniom, zaczynając od tych pierwszych. Zwróćmy uwagę na to, że poza

wartością sequence number widnieje również sequence number (raw), co widać na Rysunku 5.

Czym są więc te wartości sekwencji? Zaczniemy od tego, że do analizy pakietów praktycznie nie korzystamy z „surowego” numeru sekwencji (sequence number (raw)). Zamiast tego używamy tzw. relatywnego numeru sekwencyjnego, który jest wyliczany przez aplikację Wireshark. Warto zaznaczyć, że relatywny numer sekwencyjny nie występuje w fizycznej formie w samym protokole TCP – jest to jedynie ułatwienie wprowadzone przez aplikację Wireshark.

Prawdziwy numer sekwencyjny, jak widzimy w pierwszym pakiecie, jest dużą, losową liczbą całkowitą. Losowość ta ma na celu zwiększenie bezpieczeństwa i zapobieganie kolizjom pakietów. Warto dodać, że trzeci pakiet (drugi pakiet wysłany przez klienta) ma ten sam numer sekwencyjny zwiększony o 1.

Kolejne numery sekwencyjne są generowane w ten sposób, że do poprzedniego numeru sekwencyjnego dodajemy ilość bajtów wysłanych w poprzednim pakiecie (nie chodzi o sumaryczną długość całej ramki danych w kolumnie length, ale o dane zawarte w samym pakiecie TCP pokazane jako Len=0 w pierwszych dwóch pakietach SYN).

Mały wyjątek od tej reguły stanowi właśnie pakiet SYN (oraz FIN, jak zobaczymy później). Mimo że nie przenoszą one żadnych danych (Len=0), to kolejny pakiet jest wysyłany z numerem sekwencyjnym zwiększonym o 1. Jest to tak zwany bajt widmo (ang. *ghost byte*), który nie jest fizycznie wysyłany, ale jest wliczany do następnego numeru sekwencyjnego.

Spójrzmy, jak kolejne wartości sequence number zwiększają się zarówno po stronie klienta, jak i serwera.

8.304333877	58250	58250 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1529650600 TSecr=0 WS=128	80	TCP: 58250 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1529650600 TSecr=0 WS=128
8.319159675	58250	80 → 58250 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM TSval=2049743639 TSecr=1529650600	80	TCP: 80 → 58250 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM TSval=2049743639 TSecr=1529650600
8.319278570	58250	58250 → 80 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=1529650615 TSecr=2049743639	80	TCP: 58250 → 80 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=1529650615 TSecr=2049743639

Rysunek 4. Nawiązywanie połączenia

Source	Destination	Protocol	Length	Sequence Number (raw)	Sequence Number	Info
10.11.0.25	216.239.38.120	TCP	74	731963153	0	58250 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1529650600 TSecr=0 WS=128
216.239.38.120	10.11.0.25	TCP	74	517790060	0	80 → 58250 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM TSval=2049743639 TSecr=1529650600
10.11.0.25	216.239.38.120	TCP	66	731963154	1	58250 → 80 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=1529650615 TSecr=2049743639
10.11.0.25	216.239.38.120	HTTP	144	731963154	1	GET / HTTP/1.1

Rysunek 5. Widok aplikacji Wireshark po dodaniu kolumn sequence number oraz sequence number (raw)

Source	Destination	Protocol	Length	Sequence Number (raw)	Sequence Number	TCP Segment Len	Info
10.11.0.25	216.239.38.120	TCP	74	731963153	0	0	58250 → 80 [SYN] Seq=0 Win=64240
10.11.0.25	216.239.38.120	TCP	66	731963154	1	0	58250 → 80 [ACK] Seq=1 Ack=1 Win=64256
10.11.0.25	216.239.38.120	HTTP	144	731963154	1	78	GET / HTTP/1.1
10.11.0.25	216.239.38.120	TCP	66	731963232	79	0	58250 → 80 [ACK] Seq=79 Ack=2801
10.11.0.25	216.239.38.120	TCP	66	731963232	79	0	58250 → 80 [ACK] Seq=79 Ack=5601
10.11.0.25	216.239.38.120	TCP	66	731963232	79	0	58250 → 80 [ACK] Seq=79 Ack=11201
10.11.0.25	216.239.38.120	TCP	66	731963232	79	0	58250 → 80 [ACK] Seq=79 Ack=14001
10.11.0.25	216.239.38.120	TCP	66	731963232	79	0	58250 → 80 [ACK] Seq=79 Ack=15401
10.11.0.25	216.239.38.120	TCP	66	731963232	79	0	58250 → 80 [ACK] Seq=79 Ack=18201
10.11.0.25	216.239.38.120	TCP	66	731963232	79	0	58250 → 80 [ACK] Seq=79 Ack=21001
10.11.0.25	216.239.38.120	TCP	66	731963232	79	0	58250 → 80 [ACK] Seq=79 Ack=21850
10.11.0.25	216.239.38.120	TCP	66	731963232	79	0	58250 → 80 [FIN, ACK] Seq=79 Ack=21851
10.11.0.25	216.239.38.120	TCP	66	731963233	80	0	58250 → 80 [ACK] Seq=80 Ack=21851

Rysunek 6. Wartości sequence number po stronie klienta

Source	Destination	Protocol	Length	Sequence Number (raw)	Sequence Number	TCP Segment Len	Info
216.239.38.1...	10.11.0.25	TCP	74	517790060	0	0	80 → 58250 [SYN, ACK] Seq=0 Ack=1
216.239.38.1...	10.11.0.25	TCP	66	517790061	1	0	80 → 58250 [ACK] Seq=1 Ack=79 Win=
216.239.38.1...	10.11.0.25	TCP	2866	517790061	1	2800	80 → 58250 [PSH, ACK] Seq=1 Ack=7
216.239.38.1...	10.11.0.25	TCP	2866	517792861	2801	2800	80 → 58250 [PSH, ACK] Seq=2801 Ac
216.239.38.1...	10.11.0.25	TCP	2866	517795661	5601	2800	80 → 58250 [PSH, ACK] Seq=5601 Ac
216.239.38.1...	10.11.0.25	TCP	2866	517798461	8401	2800	80 → 58250 [PSH, ACK] Seq=8401 Ac
216.239.38.1...	10.11.0.25	TCP	2866	517801261	11201	2800	80 → 58250 [PSH, ACK] Seq=11201 A
216.239.38.1...	10.11.0.25	TCP	1466	517804061	14001	1400	80 → 58250 [ACK] Seq=14001 Ack=79
216.239.38.1...	10.11.0.25	TCP	2866	517805461	15401	2800	80 → 58250 [PSH, ACK] Seq=15401 A
216.239.38.1...	10.11.0.25	TCP	2866	517808261	18201	2800	80 → 58250 [PSH, ACK] Seq=18201 A
216.239.38.1...	10.11.0.25	HTTP	915	517811061	21001	849	HTTP/1.1 200 OK (text/html)
216.239.38.1...	10.11.0.25	TCP	66	517811910	21850	0	80 → 58250 [FIN, ACK] Seq=21850 A

Rysunek 7. Wartości sequence number po stronie serwera

W obu przypadkach zwróćmy uwagę zarówno na kolumnę sequence number, jak i TCP Segment Len. Numer sekwencyjny zawsze zwiększa się o długość segmentu TCP (danych wysyłanych w pakiecie TCP). W przypadku pakietów wysyłanych przez klienta większość z nich to pakiety ACK, które nie przenoszą dodatkowych danych, stąd ich wartości numerów sekwencyjnych się nie zwiększa. Wyjątkiem są wspomniane pakiety SYN i FIN.

Skoro już opieramy się na wartości Len, warto wspomnieć, skąd wiadomo, jakiej długości jest segment TCP. Jeśli przyjrzymy się Rysunkowi 1, gdzie jest schemat pakietu TCP, to zauważymy, że informacji o długości danych tam nie ma i jest to zgodne z prawdą. Długość segmentu danych TCP jest wyliczona na podstawie długości podanej w ramce IP – (minus) rozmiar nagłówka TCP [11].

I Potwierdzenie (ACKnowledgements)

Z numerami sekwencyjnymi idą w parze potwierdzenia. Dzięki nim strony komunikacji wiedzą, które z wiadomości dotarły do odbiorcy, a które wymagają retransmisji.

Pokrótkie wygląda to tak, że wartość acknowledge mówi drugiej stronie komunikacji, jakiego następnego numeru sequence się spodziewa, co jednocześnie informuje o tym, ile bajtów danych zostało poprawnie dostarczonych. Spójrzmy bliżej na wartości acknowledge.

Na Rysunku 8 widzimy powiązanie numeru potwierdzenia (ang. *acknowledgement number*) z numerem sekwencyjnym (ang. *sequence number*). Dokładniej rzecz ujmując, wartość ACK informuje nas o spodziewanej wartości numeru sekwencyjnego kolejnego pakietu lub bardziej precyzyjnie, ile bajtów danych zostało poprawnie odebranych.

Na rysunku przedstawiono również dwa dość specyficzne scenariusze: kolejne pakiety serwera wysyłane przed odesłaniem pakietu ACK oraz pakiet ACK potwierdzający 2 pakiety jednocześnie.

Na początku widzimy znaną już sekwencję nawiązywania połączenia. Acknowledgement number pierwszego pakietu to 0, dlatego że kolejny pakiet, którego się spodziewamy od serwera, to właśnie pakiet SYN/ACK z numerem sekwencyjnym 0. Następnie widzimy ACK o wartości 1 (ze względu na wielokrotnie wspomniany ghost byte).

Source	Destination	Sequence Number	TCP Segment Len	Acknowledgment Number	Info
10.11.0.25	216.239.38.120	0	0	0	58250 → 80 [SYN] Seq=0
216.239.38.1...	10.11.0.25	0	0	1	80 → 58250 [SYN, ACK] S
10.11.0.25	216.239.38.120	1	0	1	58250 → 80 [ACK] Seq=1
10.11.0.25	216.239.38.120	1	78	1	GET / HTTP/1.1
216.239.38.1...	10.11.0.25	1	0	79	80 → 58250 [ACK] Seq=1
216.239.38.1...	10.11.0.25	1	2800	79	80 → 58250 [PSH, ACK] S
216.239.38.1...	10.11.0.25	2801	2800	79	80 → 58250 [PSH, ACK] S
216.239.38.1...	10.11.0.25	5601	2800	79	80 → 58250 [PSH, ACK] S
216.239.38.1...	10.11.0.25	8401	2800	79	80 → 58250 [PSH, ACK] S
10.11.0.25	216.239.38.120	79	0	2801	58250 → 80 [ACK] Seq=79
10.11.0.25	216.239.38.120	79	0	5601	58250 → 80 [ACK] Seq=79
10.11.0.25	216.239.38.1...	79	0	11201	58250 → 80 [ACK] Seq=7
216.239.38.1...	10.11.0.25	11201	2800	79	80 → 58250 [PSH, ACK] S
216.239.38.1...	10.11.0.25	14001	1400	79	80 → 58250 [ACK] Seq=14
10.11.0.25	216.239.38.120	79	0	14001	58250 → 80 [ACK] Seq=79
10.11.0.25	216.239.38.120	79	0	15401	58250 → 80 [ACK] Seq=79

Rysunek 8. Komunikacja z informacją o wartości acknowledge oraz sequence number

Po nawiązaniu połączenia i wysłaniu żądania widzimy wiele pakietów wysyłanych przez serwer, które nie czekają na odpowiadające im potwierdzenia w postaci pakietu ACK. Takie zachowanie jest pożądane. Zastanówmy się, z czego ono wynika.

Wysyłane pakiety są ograniczone pod względem rozmiaru (nie możemy wysłać pakietu o dowolnie dużym rozmiarze), dlatego ulegają one tzw. segmentacji. Gdybyśmy mogli wysłać pakiety o nieograniczonej długości, jeden pakiet mógłby zmonopolizować całą komunikację. Dzięki podziałowi pakietu na mniejsze części umożliwiamy wysyłanie i odbieranie innych pakietów.

Wracając do tematu wysyłania pakietów bez czekania na odpowiadające im pakiety ACK, zastanówmy się, co by było, gdybyśmy za każdym razem czekali na potwierdzenie pakietu.

RTT (Round Trip Time)

Obie strony komunikacji znajdują się w pewnej odległości od siebie. Odległość ta powoduje opóźnienia w komunikacji, które określa się wartością RTT (ang. *Round Trip Time*) [4]. Jest to czas „podróży” pakietu w obie strony (od nadawcy do odbiorcy i z powrotem). Wartość ta może się zmieniać w trakcie trwania połączenia – w różnych momentach RTT może rosnąć lub maleć w zależności od wielu czynników, jak obciążenie routerów czy medium komunikacji (zakłócenia przy komunikacji bezprzewodowej). Pomimo tych zmian możemy oszacować wstępną wartość RTT podczas nawiązywania połączenia TCP.

Na Rysunku 9 widzimy (w kolumnie *time since first frame*), że RTT wynosi około 24ms. Z wcześniejszych rysunków wynika, że rozmiar danych w pakiecie wynosi 2800 bajtów. Gdybyśmy za każdym musieli czekać 24ms po wysłaniu 2800 bajtów na pakiet ACK, to maksymalnie moglibyśmy przesłać około 117 kB w ciągu sekundy. Jest to skrajnie nieefektywne, zwłaszcza biorąc pod uwagę coraz powszechniejsze wykorzystanie łącz o przepustowości 1 Gb/s, co odpowiada około 125 MB/s.

Widzimy więc, że możliwość wysyłania kolejnych pakietów bez czekania na ACK może znacznie przyspieszyć transmisję danych.

Retransmisje i selektywne retransmisje

Wiemy już, jak działają numery sekwencyjne oraz potwierdzenia, więc najwyższy czas omówić ich praktyczne zastosowanie.

Może się bowiem zdarzyć, że jeden z pakietów serii nie dotrze do odbiorcy lub dotrze z dużym opóźnieniem. W takim przypadku obie strony muszą być w stanie odpowiednio zareagować – strona, która nie otrzymała pakietu, nie wyśle potwierdzenia, lub wyśle informację o brakujących pakietach, natomiast nadawca musi te pakiety ponownie wysłać.

Właśnie w ten sposób protokół TCP, w odróżnieniu od UDP, gwarantuje dostarczenie wszystkich pakietów lub przynajmniej informację o ich niedostarczeniu.

Przeanalizujemy kilka scenariuszy retransmisji danych.

Retransmisje ze względu na timeout [12]

Podstawowym mechanizmem wywołującym retransmisję jest upływanie określonego interwału czasu (RTO – retransmission timeout). Po wysłaniu pakietu nadawca rozpoczyna odliczanie (w zależności od implementacji, zazwyczaj jest to wielokrotność czasu RTT). Jeśli po upływie tego czasu nie otrzyma potwierdzenia odbioru, ponownie wysła pakiet.

Warto zaznaczyć, że w podstawowym mechanizmie potwierdzeń pakietów odbiorca nie ma możliwości wskazania, który dokładnie pakiet został zgubiony, ponieważ protokół wykorzystuje jedną wartość ACK. Przeanalizujemy to na przykładzie.

Załóżmy, że nadawca wysła 100 bajtów w 10 pakietach po 10 bajtów każdy. W tej serii pakietów założmy, że bajty 51-60 nie dotarły do odbiorcy, podczas gdy wszystkie pozostałe zostały poprawnie dostarczone.

Time	Time since first frame in this TCP stream	Source	Destination	Info
11.497404595	0.000000000	10.11.0.25	216.239.38.120	41876 → 80 [SYN] Seq=0 Win=64240 L
11.521405468	0.024000873	216.239.38.120	Source address	80 → 41876 [SYN, ACK] Seq=0 Ack=1

Rysunek 9. RTT na podstawie SYN oraz SYN/ACK

No.	Time since previous Time	Source	Destination	Protocol	Length	Info
9	0.000000000	6.883342162	10.11.0.25	TCP	54	12347 → 80 [SYN] Seq=0
10	0.014898449	6.898240611	216.239.38.120	TCP	58	80 → 12347 [SYN, ACK] S
11	0.033049606	6.931290217	10.11.0.25	TCP	54	12347 → 80 [ACK] Seq=1
12	0.037336572	6.968626789	10.11.0.25	HTTP	93	GET / HTTP/1.1
13	0.016625159	6.985251948	216.239.38.120	TCP	56	80 → 12347 [ACK] Seq=1
14	0.277382125	7.262634073	10.11.0.25	TCP	1126	80 → 12347 [PSH, ACK] S
15	0.000000273	7.262634346	216.239.38.120	TCP	1126	80 → 12347 [PSH, ACK] S
16	0.000000074	7.262634420	216.239.38.120	TCP	1126	80 → 12347 [PSH, ACK] S
17	0.000000072	7.262634492	216.239.38.120	TCP	1126	80 → 12347 [PSH, ACK] S
18	0.000062713	7.262697205	216.239.38.120	TCP	1126	HTTP/1.1 200 OK [TCP P
19	0.204605250	7.467302455	216.239.38.120	TCP	590	[TCP Retransmission] 80
20	0.512259913	7.979562368	216.239.38.120	TCP	590	[TCP Retransmission] 80
22	0.973288099	8.952850467	216.239.38.120	TCP	590	[TCP Retransmission] 80
23	2.099184421	11.052034888	216.239.38.120	TCP	590	[TCP Retransmission] 80

Rysunek 10. Przykład retransmisji TCP ze względu na timeout

Odbiorca po otrzymaniu pierwszych 50 bajtów, a następnie kolejnych 40 (od 61-100) może jedynie wysłać pakiet ACK z wartością 51. Wysłanie większej wartości ACK oznaczałoby dla nadawcy, że wszystkie bajty do tej wartości zostały poprawnie odebrane. W efekcie nadawca musi „cofnąć się” i wysłać ponownie wszystkie bajty od 51 do 100.

W następnej sekcji (SACK) omówimy rozszerzenie TCP, które pozwala na dokładniejsze wskazywanie brakujących pakietów.

Przykład takiej retransmisji ze względu na timeout odbywa się w następujący sposób.

Na początku widzimy standardowe nawiązanie połączenia TCP. Jak widzimy, czas RTT to około 15ms (time since previous frame pakietu numer 10), po czym następuje wysłanie żądania HTTP.

Serwer odpowiada serią pakietów TCP, jednak żaden z pakietów nie jest akceptowany przez odbiorcę. Po około 200ms (time pakietu numer 19 – time pakietu numer 14) od pierwszego niezaakceptowanego pakietu widzimy retransmisję danych, tym razem wysyłany segment danych jest mniejszy (593 bajtów w pakiecie retransmitowanym i 1126 w pakiecie oryginalnym). W ten sposób wysyłający stara się być mniej agresywny w wysyłce danych. Jako że odbierający nadal nie wysłał pakietu ACK, kolejno po około 500ms, 1000ms i 2000ms ten sam pakiet jest retransmitowany z nadzieją, że ACK w końcu zostanie odesłane.

Wybiórcze potwierdzenia (SACK)

Wybiórcze potwierdzenia – SACK (Selective Acknowledgements) – to mechanizm, który pozwala odbiorcy danych znacznie precyzyjniej informować nadawcę, które dane zostały utracone w trakcie transmisji.

SACK nie jest standardowym polem nagłówka TCP. Jest to opcjonalne rozszerzenie dodawane do pola Options nagłówka tylko wtedy, kiedy potrzeba użycia SACK ma miejsce i obie strony komunikacji go obsługują (o czym można się dowiedzieć po wymianie pakietów SYN przy nawiązywaniu połączenia).

Wracając do naszego przykładu z wysyłaniem 100 bajtów w pakietach po 10 bajtów każdy. Po wysłaniu danych 41-50 odbiorca odeśle potwierdzenie z wartością ACK=51. Kolejny pakiet nie dociera do odbiorcy, ale odbiorca otrzymuje pakiety 61-70, 71-80 itd.

W takiej sytuacji odbiorca kolejno wysła potwierdzenia:

- » <ACK = 51, pusty SACKL pusty SACKR>
- » <ACK=51, SACKL=61, SACKR=71>
- » <ACK=51, SACKL=61, SACKR=81>
- » itd.

Pole SACKL odpowiednio wskazuje początek otrzymanego bloku danych (tzw. lewy brzeg), a SACKR jego koniec (prawy brzeg).

Dzięki temu nadawca może dokładnie określić, które dane nie dotarły do odbiorcy (różnica między SACKL a ACK), i zdecydować o ponownym wysłaniu tylko brakujących pakietów, oszczędzając w ten sposób transfer – Rysunek 11.

Widzimy, że serwer wysła kolejno pakiety z bajtami:

1. 1-2800
2. 2801-5600
3. 5601-8400
4. 8401-11200

Następnie klient odsyła standardowe potwierdzenia pakietów 1 (na rysunku pakiet numer 18), 2 (na rysunku pakiet numer 19).

No.	Protocol	Length	Sequence Number	Next Sequence Number	Acknowledgment Number	Info
13	TCP	66	1	1		97 80 → 42154 [ACK] Seq=1 Ack=97 Win=268800 Len=0 TSval:
14	TCP	2866	1	2801		97 80 → 42154 [PSH, ACK] Seq=1 Ack=97 Win=268800 Len=2800
15	TCP	2866	2801	5601		97 80 → 42154 [PSH, ACK] Seq=2801 Ack=97 Win=268800 Len=2800
16	TCP	2866	5601	8401		97 80 → 42154 [PSH, ACK] Seq=5601 Ack=97 Win=268800 Len=2800
17	TCP	2866	8401	11201		97 80 → 42154 [PSH, ACK] Seq=8401 Ack=97 Win=268800 Len=2800
18	TCP	66	97	97		2801 42154 → 80 [ACK] Seq=97 Ack=2801 Win=69760 Len=0 TSval:
19	TCP	66	97	97		5601 42154 → 80 [ACK] Seq=97 Ack=5601 Win=75392 Len=0 TSval:
20	TCP	2866	11201	14001		97 80 → 42154 [PSH, ACK] Seq=11201 Ack=97 Win=268800 Len=2800
21	TCP	78	97	97		5601 [TCP Dup ACK 19#1] 42154 → 80 [ACK] Seq=97 Ack=5601 Win=0 Len=0
22	TCP	78	97	97		5601 [TCP Dup ACK 19#2] 42154 → 80 [ACK] Seq=97 Ack=5601 Win=0 Len=0
[Checksum Status: Unverified]						
Urgent Pointer: 0						
Options: (24 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps, No-Operation						
> TCP Option - No-Operation (NOP) > TCP Option - No-Operation (NOP) > TCP Option - Timestamps: TSval 44785142, TSecr 3437603572 > TCP Option - No-Operation (NOP) > TCP Option - No-Operation (NOP) ✓ TCP Option - SACK 8401-11201 - Kind: SACK (5) - Length: 10 - left edge = 8401 (relative) - right edge = 11201 (relative)						
<pre> 0000 9e 05 d6 53 89 8f 74 e5 f9 86 f9 0c 08 00 45 00 0010 00 40 6e ce 40 00 40 06 c2 5e 0a 0b 00 19 d8 00 0020 26 78 a4 aa 00 50 d1 bd e7 17 fc fa 95 d5 b0 00 0030 02 78 25 15 00 00 01 01 08 0a 02 ab 5d f6 cc 00 0040 aa f4 01 01 05 0a fc fa a0 c5 fc fa ab b5 </pre>						

Rysunek 11. Przykład komunikacji z wykorzystaniem SACK

SACK zostaje użyty pierwszy raz w pakiecie numer 21. Wireshark identyfikuje ten pakiet jako Dup ACK (Duplicate ACK), ponieważ widzi ten sam numer ACK (5601), jednak nie jest to taki sam pakiet. W szczegółach pakietu zauważamy, że jest ustawiona opcja SACK z wartościami SACKL=8401 oraz SACKR=11201. Oznacza to, że ten pakiet potwierdza odebranie pakietu 4 (na rysunku pakiet 17), jednocześnie informując, że aplikacja nie otrzymała pakietu 3 (na rysunku pakiet numer 16), o czym świadczy przerwa między wartością ACK – 5601 a SACKL – 11201. W późniejszej komunikacji serwer ponownie wyśle zagubiony pakiet.

I Fast retransmissions

Szybka retransmisja to kolejny wariant retransmisji. Od standardowej retransmisji różni się tym, że wysyłający nie czeka na upływanie odpowiedniego czasu. Zamiast tego po otrzymaniu 3 zduplikowanych (czyli w sumie 4) pakietów ACK dla tego samego pakietu natychmiast rozpoczyna retransmisję danych. Dla wysyłającego zduplikowane pakiety ACK oznaczają, że kolejne pakiety nie dotarły prawidłowo i należy je wysłać ponownie.

I Okno danych [13]

Tematem wysyłania wielu pakietów TCP bez czekania na ich potwierdzenie jest nieodłącznie związany z oknem danych (ang. *window*).

W protokole TCP obie strony komunikacji informują o rozmiarze tak zwanego okna danych. Jest to nic innego jak bufor zarezerwowany na potrzeby transmisji TCP/IP po stronie odbiorcy.

Aby zrozumieć, dlaczego okno danych jest istotne, wyobraźmy sobie scenariusz, w którym jedna strona komunikacji wysyła bardzo

dużą ilość danych (np. 1 GiB na jednostkę czasu), podczas gdy ich przetwarzanie po stronie odbiorcy jest znacznie wolniejsze (np. 1 KiB na jednostkę czasu). W takim przypadku implementacja stosu TCP po stronie odbiorcy musiałaby zarezerwować ogromną ilość pamięci, co szybko mogłoby doprowadzić do jej wyczerpania – zwłaszcza przy wielu jednoczesnych podobnych transmisjach.

Aby temu zapobiec, strony informują się wzajemnie, jak duży bufor danych został zaalokowany do obsługi komunikacji. W kolejnych pakietach aktualizują informację o dostępnej przestrzeni w oknie.

Żadna ze stron nie może wysłać ilości danych przepełniających dostępny bufor bez otrzymania pakietu ACK. Wraz z nadejściem tego pakietu nadawca otrzymuje nową informację o dostępnej przestrzeni w oknie, którą może ponownie wypełnić.

Warto zaznaczyć, że jeśli nadawca wyśle więcej danych, niż pozwala na to okno, nadmiarowe pakiety zostaną odrzucone. Przepełnienie bufora nie jest więc w pełni zależne od klienta – to odbiorca kontroluje, ile danych przyjmuje.

Zobaczmy przykład tego zachowania w praktyce:

Na Rysunku 12 widzimy kilka pakietów PSH, po których następuje sekwencja pakietów ACK potwierdzających ich odebranie. Z każdym kolejnym pakietem ACK obserwujemy zwiększenie rozmiaru okna.

Jest to „zdrowa” sytuacja – to znaczy, że klient wystarczająco szybko przetwarza odebrane dane. Poprzez zwiększenie okna zachęca on drugą stronę do wysyłania jeszcze większej ilości danych bez konieczności czekania na potwierdzenie każdego segmentu.

Gorsza sytuacja ma miejsce wtedy, gdy odbiorca pakietów przetwarza dane zbyt wolno. W takim przypadku zauważymy, że wraz z kolejnymi pakietami ACK rozmiar okna będzie się zmniejszał.

No.	Time	Source	Destination	Length	Acknowledgment Number	Calculated window size	Info
20	4.515587539	216.239.38.120	10.11.0.25	66	79	268800	80 → 42524 [ACK] Seq=1 Ack=79 Win=
21	4.572430972	216.239.38.120	10.11.0.25	2866	79	268800	80 → 42524 [PSH, ACK] Seq=1 Ack=79
22	4.572431758	216.239.38.120	10.11.0.25	2866	79	268800	80 → 42524 [PSH, ACK] Seq=2801 Ack=
23	4.572432030	216.239.38.120	10.11.0.25	2866	79	268800	80 → 42524 [PSH, ACK] Seq=5601 Ack=
24	4.572432277	216.239.38.120	10.11.0.25	2866	79	268800	80 → 42524 [PSH, ACK] Seq=8401 Ack=
25	4.572543147	10.11.0.25	216.239.38.120	66	2801	69760	42524 → 80 [ACK] Seq=79 Ack=2801 W
26	4.572604886	10.11.0.25	216.239.38.120	66	5601	75392	42524 → 80 [ACK] Seq=79 Ack=5601 W
27	4.572627698	10.11.0.25	216.239.38.120	66	8401	80896	42524 → 80 [ACK] Seq=79 Ack=8401 W
28	4.572650972	10.11.0.25	216.239.38.120	66	11201	78976	42524 → 80 [ACK] Seq=79 Ack=11201

Rysunek 12. Zmiana okna TCP w czasie

25045	10.11.0.25	216.239.38.120	19983	79	42524 → 80 [FIN, ACK] Seq=79 Ac
38282	216.239.38.120	10.11.0.25	80	19983	80 → 42524 [FIN, ACK] Seq=19983
75022	10.11.0.25	216.239.38.120	19984	80	42524 → 80 [ACK] Seq=80 Ack=199

Rysunek 13. Zakończenie połączenia TCP

No.	Time	Source	Destination	Protocol	Length	Info
3	0.000059364	127.0.0.1	127.0.0.1	TCP	74	60984 → 8123 [SYN] Seq=0
4	0.000067116	127.0.0.1	127.0.0.1	TCP	54	8123 → 60984 [RST, ACK]

Rysunek 14. Próba utworzenia połączenia z zamkniętym portem – pakiet RST

W skrajnym przypadku serwer wstrzyma wysyłanie danych, gdy wykryje, że dalsza transmisja przekroczyłaby dostępne okno odbiorcy.

I Zrywanie połączenia [14]

Skoro wiemy już, jak przebiega nawiązywanie połączenia oraz transmisja danych, czas przyjrzeć się temu, jak połączenie ulega zakończeniu.

Istnieje kilka metod zamykania połączenia. Najbardziej eleganckim sposobem jest wysłanie sekwencji pakietów FIN i ACK.

Zobaczmy więc, jak wygląda zakończenie połączenia w praktyce.

Standardowo na schematach zakończenie połączenia TCP przedstawia się jako sekwencja dwóch pakietów FIN, z których każdy jest potwierdzany dedykowanym pakietem ACK, ale w tym przypadku zarówno ACK, jak i FIN ze strony serwera zostały scalone w jeden pakiet. Podobnie ma się sprawa z nawiązywaniem połączenia, gdzie pakiet SYN/ACK to dwa pakiety scalone w jeden.

Na Rysunku 13 widzimy pierwszy pakiet FIN pochodzący ze strony klienta. Pakiet ten ma numer sekwencyjny 79. Od tego momentu klient nie może wysłać więcej danych, ale może je nadal odbierać i potwierdzać. Kolejno widzimy odpowiedź serwera, który również sygnalizuje zamknięcie połączenia po swojej stronie i potwierdza pakiet FIN klienta. Ostatni pakiet to potwierdzenie pakietu FIN serwera przez klienta. Warto dodać, że potwierdzenie to jest jedynym pakietem, na który może pozwolić sobie klient. W przeciwnym razie serwer odpowiedziałby pakietem RST.

Pakiet RST również może być używany do zakończenia połączenia, jednak jego zastosowanie różni się od standardowego zamykania sesji TCP przy użyciu FIN.

I RST

Najpopularniejszym zastosowaniem pakietów RST w normalnym użytku (niezwiązanym z błędami transmisji) jest natychmiastowe zamykanie połączenia TCP. Ich głównym przeznaczeniem jest jednak sygnalizowanie problemów z połączeniem (jak np. próba komunikacji z zamkniętym portem).

Użycie flagi RST zamiast standardowego procesu z wykorzystaniem pakietów FIN ma pewne zalety. Przede wszystkim pakiety RST nie wymagają potwierdzenia ich odbioru, a implementacje protokołu TCP po otrzymaniu takiego pakietu powinny natychmiast uznać połączenie za zamknięte.

W przeciwieństwie do standardowego procesu zamykania połączenia strony nie muszą utrzymywać maszyny stanów (odpowiednio zapamiętanie, czy pakiet FIN został wysłany, czy został potwierdzony, czy druga strona odesłała pakiet itp.).

Nie znaczy to jednak, że kończenie połączenia w ten sposób jest lepsze. Zakończenie połączenia przez wysłanie pakietu RST nie gwarantuje dostarczenia danych wysłanych przez drugą stronę (co miałyby miejsce w przypadku wysłania pakietu FIN).

Warto wspomnieć, że w standardowym procesie zamykania połączenia mogłoby się wydawać, że jeśli potwierdzenie pakietu FIN nigdy nie zostanie wysłane, implementacja TCP po drugiej stronie komunikacji będzie czekać w nieskończoność (i retransmitować pakiety FIN). Tak oczywiście nie jest – implementacje po kilku nieudanych retransmisjach danych uznają połączenie za zamknięte.

I Wykorzystanie RST a błędy połączenia

Zobaczmy kilka innych scenariuszy, w których to wysyłany jest pakiet RST. Spróbujmy np. utworzyć połączenie na porcie, który jest zamknięty, co ilustruje Rysunek 14.

Jak widzimy, w odpowiedzi na pakiet SYN od razu otrzymujemy pakiet RST. Takie zachowanie jest często wykorzystywane podczas skanowania komputerów w poszukiwaniu otwartych portów (choć istnieją również alternatywne metody skanowania).

Pakiet RST może być również wysyłany przez zapory sieciowe (firewall), chociaż często zamiast pakietu RST firewall po prostu nie odsyła żadnego pakietu.

Pakiet RST zobaczymy również w przypadku pojawienia się pakietów, które nijak nie pasują do obecnego połączenia. Mogą na to wskazywać chociażby pakiety, których wartości acknowledgement i sequence znacznie odbiegają od tych obecnie obserwowanych w połączeniu.

I TTL

TTL nie należy do warstwy TCP, ale do warstwy IP. Znajomość tej wartości może nam pomóc w diagnozowaniu problemów związanych z połączeniami.

Kiedy tworzony jest pakiet, przypisywany jest mu numer TTL (ang. *Time To Live*). Zazwyczaj przyjmuje on wartość 64, 128 lub 255. Pole to zostało wprowadzone, aby pakiet „wygasł”, gdyby w sieci znalazła się pętla urządzeń, które nieustannie przesyłają do siebie ten sam pakiet. Każde przejście pakietu przez router zmniejsza wartość TTL o 1. Kiedy wartość TTL osiągnie zero, pakiet jest odrzucany.

Jeżeli zobaczymy w komunikacji TCP błąd, np. nieoczekiwany pakiet RST, to nie oznacza jeszcze, że serwer docelowy jest odpowiedzialny za jego wysłanie. Błąd może wystąpić w dowolnym miejscu na ścieżce połączenia (przypomnijmy sobie, że zanim pakiet trafi do odbiorcy, zazwyczaj przechodzi przez wiele routerów). Wartość TTL może pomóc nam określić w przybliżeniu, w którym miejscu sieci wystąpił błąd.

Jeżeli więc zobaczymy przychodzący pakiet RST z TTL równym 64, 128, 255, to istnieje prawdopodobieństwo, że został on wysłany przez nasz własny router (wartość TTL nie zmieniła się od tej domyślnie ustawionej). Podobnie, jeśli zobaczymy, że wartość jest mniejsza o około 10 od domyślnych, to możemy przypuszczać, że pakiet został wysłany przez serwis (lub jeden z elementów całej infrastruktury serwisu) [15].

Należy jednak zwrócić uwagę, że nie jesteśmy w stanie w 100% określić, które urządzenie odesłało pakiet, ponieważ zależy to od początkowych wartości TTL, które mogą się różnić w zależności od urządzenia.

I SZCZEGÓŁY UDP

Jak już wspomnieliśmy, protokół UDP jest znacznie prostszy (a przez to również bardziej zawodny) niż protokół TCP. Dlatego nie musimy w nim omawiać mechanizmów retransmisji pakietów – UDP nie weryfikuje, czy pakiet dotarł do odbiorcy. Mimo to, dzięki swojej prostocie, UDP oferuje funkcjonalność, której TCP nie obsługuje – multicast i broadcast.

Pakiety multicast są kierowane do grupy odbiorców subskrybujących dany adres, a broadcast trafia do wszystkich urządzeń w sieci LAN. UDP może wykorzystać tę funkcjonalność, ponieważ nie wymaga nawiązania połączenia między nadawcą a odbiorcą. TCP natomiast, ze względu na konieczność nawiązania połączenia, nie obsługuje ani multicasu, ani broadcastu.

Nie będziemy się jednak zagłębiać w to, jak działa multicast i broadcast w szczegółach, ponieważ skupiamy się na komunikacji między serwisami (z punktu widzenia protokołu HTTP z poprzedniego artykułu). W tym kontekście UDP ma ograniczone zastosowanie, zwłaszcza jeśli chodzi o funkcjonalność multicasu i broadcastu.

Warto jednak pamiętać, że taka funkcjonalność istnieje. Aby wysłać pakiet do wielu odbiorców, wykorzystuje się specjalne zakresy adresów IP. Dla multicasu są to adresy z zakresu 224.0.0.0 – 224.255.255.255, które są przypisane do odpowiednich grup odbiorców, a dla broadcastu – uniwersalny adres 255.255.255.255.

I NAT

Ostatnia rzecz, o której chciałbym wspomnieć, to NAT (ang. *Network Address Translation*), a dokładniej PAT (ang. *Port Address Translation*).

Prawdopodobnie czytelnik zauważył, że ten sam adres IP (np. na stronie whatsmyip.com) może być współdzielony przez niego oraz sąsiada. Być może zauważył również, że jego dostawca Internetu oferuje publiczne adresy IP. Jak to wszystko więc działa, skoro dzielimy nasze adresy IP z innymi użytkownikami w sieci? Odpowiedź na to pytanie zawiera się właśnie w zrozumieniu działania NAT.

Wspomnieliśmy już o tym zjawisku w poprzednich artykułach, jednak nie zagłębialiśmy się w szczegóły. Skoro jednak znamy już protokoły TCP i UDP, możemy przyrzeć się bliżej szczegółom działania NAT.

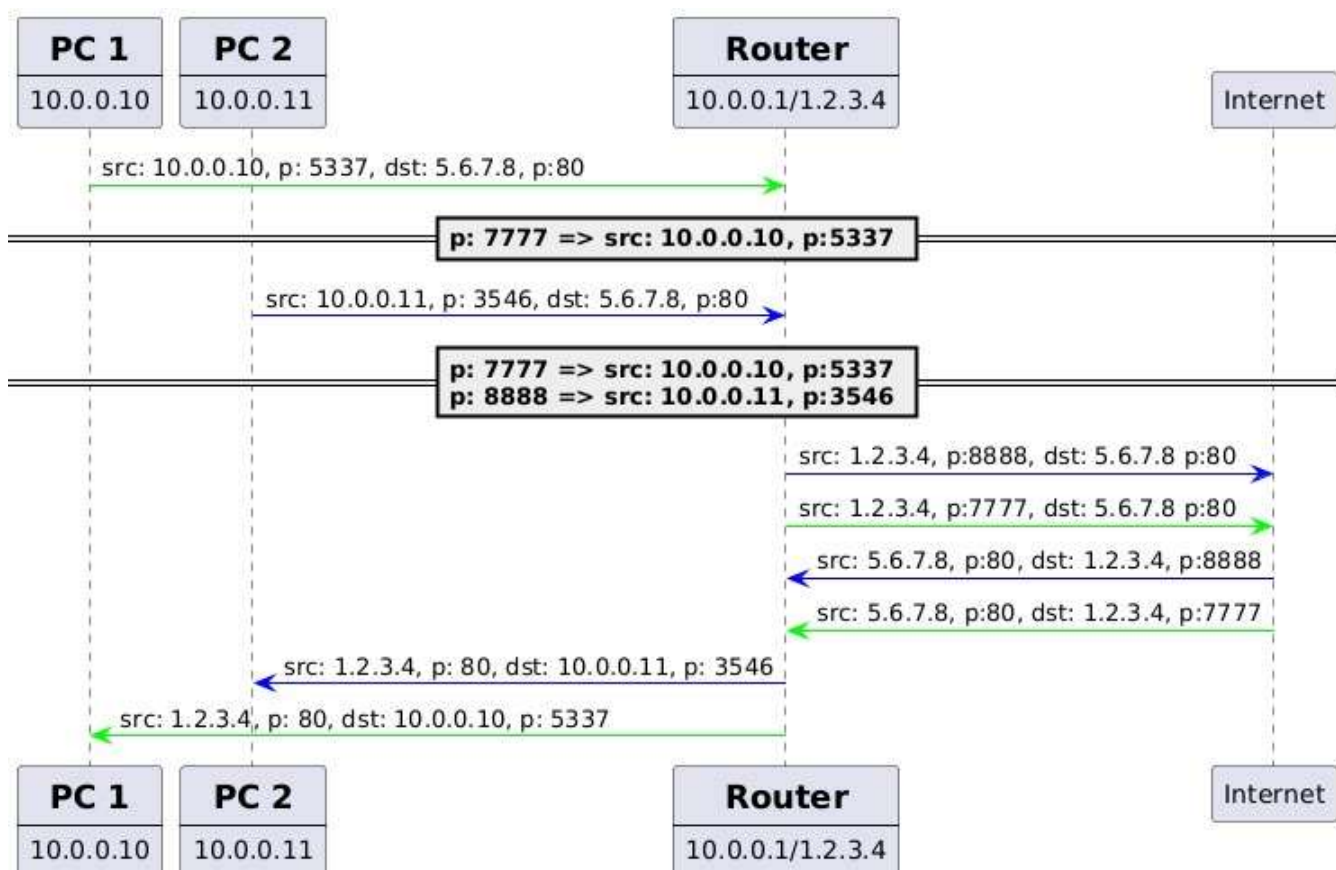
NAT, czyli Network Address Translation, to zbiór mechanizmów, które modyfikują pakiety przechodzące przez router. NAT może być statyczny oraz dynamiczny.

Stacyjny NAT działa na prostej zasadzie – adresowi IP hosta w sieci LAN przypisany jest określony publiczny adres IP. Kiedy pakiet przechodzi przez router, ten podmienia adres IP w pakiecie przed wysłaniem go dalej. Router ma więc na stałe skonfigurowane mapowanie między prywatnymi a publicznymi adresami IP.

Dynamiczny NAT działa podobnie do statycznego, z tą różnicą, że publiczny adres IP jest wybierany dynamicznie z dostępnej puli adresów, którą ma przypisaną. Gdy pakiet przechodzi przez router z sieci prywatnej do Internetu, router wybiera dostępny adres IP z puli i podmienia prywatny adres na ten właśnie wybrany. Takie dynamiczne mapowanie jest zapamiętywane do momentu zakończenia połączenia, aby umożliwić translację w obie strony (z sieci prywatnej do Internetu i z powrotem). Jednak to rozwiązanie jest obecnie rzadziej stosowane, a w jego miejsce wykorzystuje się PAT (Port Address Translation).

Jeszcze inne rodzaje NAT to source NAT (SNAT) oraz destination NAT (DNAT). DNAT omówimy na przykładzie port forwardingu, a SNAT na przykładzie PAT.

PAT (Port Address Translation) umożliwia urządzeniom w sieci LAN współdzielenie publicznego IP. PAT, jak sama nazwa wskazuje, opiera się portach, które są niczym innym jak pewną wartością cał-



Rysunek 15. Schemat działania PAT

kowitoliczbową, która identyfikuje usługę, z którą chcemy się komunikować. Ta wartość jest używana przez PAT do zapamiętania, który numer portu jest związany z którym adresem prywatnym IP.

Przyjrzyjmy się teraz przykładowemu działaniu PAT.

Na Rysunku 15 widzimy dwa hosty – PC1 i PC2 – które znajdują się w sieci prywatnej. Ich adresy IP to odpowiednio 10.0.0.10 oraz 10.0.0.11. Komunikują się one z routerem, którego adres wewnętrzny to 10.0.0.1, a zewnętrzny – 1.2.3.4.

Do routera docierają dwa pakiety danych – jeden od PC1, a drugi od PC2. Oba pakiety są adresowane do tego samego hosta w Internecie (5.6.7.8). Router musi zmodyfikować te pakiety, aby móc odróżnić odpowiedzi przychodzące od hosta 5.6.7.8 i przekierować je do odpowiednich hostów (PC1 lub PC2). W tym celu:

- » Dla pakietu od PC1 alokuje port 7777 i zapisuje w swojej pamięci zależność między tym portem a PC1 (IP 10.0.0.10, port 5337).
- » Dla pakietu od PC2 alokuje port 8888 i zapisuje zależność między tym portem a PC2 (IP 10.0.0.11, port 3546).

W pakietach wychodzących router modyfikuje adres IP źródłowy na swój własny publiczny adres IP oraz port źródłowy na wcześniej zaalokowany port, a następnie wysyła pakiety dalej.

Kiedy pakiety przychodzą do portów 7777 i 8888 routera, jest on w stanie zmodyfikować adres IP docelowy oraz port docelowy pakietu, aby wskazywały odpowiednio na PC1 i PC2 i odpowiednie dla nich porty. Informacje te są przechowywane w pamięci routera.

Jak nietrudno zauważyć, każde urządzenie w sieci LAN może mieć otwarte wiele połączeń, może się więc zdarzyć sytuacja, w której

to nasz router nie będzie miał żadnego wolnego portu, żeby utworzyć nowe mapowanie. W takiej sytuacji router będzie musiał odrzucić nasze połączenie.

Problem pojawia się, gdy jeden z hostów w sieci LAN chce udostępnić publicznie usługę, którą hostuje. Aby umożliwić taką komunikację, powstał mechanizm port forwarding. W skrócie polega on na stałym przypisaniu portu na routerze do konkretnego adresu IP i portu hosta w sieci LAN. W ten sposób pakiety przychodzące do routera trafiają następnie do odpowiedniego hosta.

I PODSUMOWANIE

Serdecznie dziękuję za wytrwanie z serią artykułów „Jak działa Internet”. Po lekturze całości powinniśmy mieć już dość dobre rozeznanie w różnych warstwach sieciowych, umieć analizować ruch sieciowy oraz skonfigurować router. Znając protokoły HTTP i TCP, jesteśmy również w stanie zaimplementować własny serwis internetowy, a wiedząc, jak działa DNS, nie powinniśmy mieć problemu z przypisaniem do niego przyjaznej nazwy domenowej.

Jeśli czytelnik chciałby zgłębić wiedzę na temat protokołu TCP, polecam kanał Chrisa Greera na YouTube, gdzie znajdziesz praktyczne przykłady różnych zachowań tego protokołu.

Jeszcze bardziej zachęcam do nauki przez praktykę – po tej serii artykułów czas stworzyć własny serwis internetowy! To doskonała okazja, aby wykorzystać zdobytą wiedzę w praktyce i lepiej zrozumieć, jak wszystkie omówione mechanizmy działają w rzeczywistych warunkach.

Bibliografia

1. <https://study-cna.com/tcpip-suite-of-protocols/>
2. <https://www.geeksforgeeks.org/user-datagram-protocol-udp/>
3. TCP/IP – <https://www.ietf.org/rfc/rfc9293.pdf>
4. RTT – <https://www.geeksforgeeks.org/what-is-rtt-round-trip-time/>
5. UDP – Introduction – <https://www.rfc-editor.org/rfc/rfc768.html#Introduction>
6. UDP vs TCP speed – <https://www.netally.com/network-performance/tcp-vs-udp/>
7. TCP motivation – <https://www.rfc-editor.org/rfc/rfc793#section-1.1>
8. TCP header – <https://www.ietf.org/rfc/rfc9293.html#name-header-format>
9. TCP connection establishment – <https://www.ietf.org/rfc/rfc9293.html#name-establishing-a-connection>
10. SACK – <https://www.rfc-editor.org/rfc/rfc2018.html>
11. IP rozmiar ramki – <https://datatracker.ietf.org/doc/html/rfc791#section-3.1>
12. TCP retransmisja ze względu na timeout – <https://blog.codefarm.me/2023/01/17/tcp-ip-tcp-timeout-and-retransmission/>
13. TCP – okno danych – https://youtu.be/x_5HIKEiViA?feature=shared
14. TCP – nawiązywanie i kończenie połączeń – <https://pasja-informatyki.pl/sieci-komputerowe/uzgadnianie-trojetapowe/>
15. TCP – analizyng RST in packets – <https://www.youtube.com/watch?v=t50Jephyw8I>
16. suma kontrolna w protokole TCP a prawdopodobieństwo wystąpienia błędu – <https://gynvael.coldwind.pl/?id=724>



DAWID PILARSKI

dawid.pilarski@pm.me

Z wykształcenia automatyk i robotyk, a z zawodu i pasji programista. Obecnie Software Engineer i Tech Lead w TomTom na wypowiedzeniu. Wkrótce starszy inżynier niskopoziomowy w NordVPN :). Wolny czas przeznaczam na zgłębianie wiedzy o bezpieczeństwie i sieciach.